

Nama:Hafidza Tsamara Zahra
NIM:254107020034
Kelas:1G-TI
Absen:11

LAPORAN JOBSHEET 5

percobaan 5.2

class faktorial11.java

```
package BruteForceDivideConquer.minggu5;

public class faktorial11 {
    int faktorialBF(int n){
        int fakto = 1;
        for(int i=1;i<=n;i++){
            fakto = fakto * i;
        }
        return fakto;
    }
    int faktorialDC(int n){
        if(n==1){
            return 1;
        }else{
            int fakto = n * faktorialDC(n-1);
            return fakto;
        }
    }
}
```

class mainFaktorial11.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class mainFaktorial11 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("masukkan nilai: ");
        int nilai = input.nextInt();

        faktorial11 fk = new faktorial11();
    }
}
```

```

        System.out.println("nilai faktorial "+nilai+"menggunakan BF:
"+fk.faktorialBF(nilai));
        System.out.println("nilai faktorial "+nilai+"menggunakan DC:
"+fk.faktorialDC(nilai));
    }
}

```

hasil output:

masukkan nilai: 5

nilai faktorial 5 menggunakan BF: 120

nilai faktorial 5 menggunakan DC: 120

pertanyaan:

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

jawaban:

1. if vs else di Divide and Conquer
 - if(n==1) : base case, menghentikan rekursi agar tidak jalan terus
 - else : rekursi, memanggil method faktorialDC(n-1) untuk menghitung faktorial bagian lebih kecil
2. Perulangan selain for di faktorialBF() Bisa diganti dengan while atau do-while, Jadi iterasi tidak harus selalu pakai for
 contoh:


```

int fakto = 1;
int i = 1;
while(i <= n){
    fakto *= i;
    i++;
}
return fakto;

```
3. fakto *= i; vs int fakto = n * faktorialDC(n-1);
 - fakto *= i; : iterasi (loop)
 - n * faktorialDC(n-1); : rekursi (memanggil diri sendiri)
4. Kesimpulan perbedaan faktorialBF() vs faktorialDC()
 - faktorialBF() : pakai loop, hitung step by step dari 1 sampai n
 - faktorialDC() : pakai rekursi, pecah masalah sampai base case, kemudian hasil dikalikan saat "naik" dari rekursi

- BF lebih sederhana, DC lebih elegan dan sesuai konsep divide & conquer

percobaan 5.3

class pangkat.java

```
package BruteForceDivideConquer.minggu5;

public class pangkat {
    int nilai, pangkat;

    pangkat(int n, int p) {
        nilai = n;
        pangkat = p;
    }

    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
        return hasil;
    }

    int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
            } else {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
            }
        }
    }
}
```

class mainPangkat.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class mainPangkat {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("masukkan jumlah elemen: ");
        int elemen = input.nextInt();
    }
}
```

```

    pangkat[] png = new pangkat[elemen];
    for(int i=0;i<elemen;i++){
        System.out.print("masukkan nilai basis elemen ke-"+(i+1)+":
");
        int basis = input.nextInt();
        System.out.print("masukkan nilai pangkat elemen
ke-"+(i+1)+": ");
        int pangkat = input.nextInt();
        png[i] = new pangkat(basis,pangkat);
    }
    System.out.println("HASIL PANGKAT BRUTEFORCE:");
    for(pangkat p : png){
        System.out.println(p.nilai+"^"+p.pangkat+":
"+p.pangkatBF(p.nilai, p.pangkat));
    }
    System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
    for(pangkat p : png){
        System.out.println(p.nilai+"^"+p.pangkat+":
"+p.pangkatDC(p.nilai, p.pangkat));
    }
}
}

```

hasil output:

masukkan jumlah elemen: 3

masukkan nilai basis elemen ke-1: 2

masukkan nilai pangkat elemen ke-1: 3

masukkan nilai basis elemen ke-2: 4

masukkan nilai pangkat elemen ke-2: 5

masukkan nilai basis elemen ke-3: 6

masukkan nilai pangkat elemen ke-3: 7

HASIL PANGKAT BRUTEFORCE:

2^3: 8

4^5: 1024

6^7: 279936

HASIL PANGKAT DIVIDE AND CONQUER:

2^3: 8

4^5: 1024

6^7: 279936

pertanyaan:

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!

3. Pada method pangkatBF()terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

jawaban

1. Perbedaan pangkatBF() dan pangkatDC()
 - pangkatBF() : Brute Force
 - Menghitung pangkat dengan loop dari 0 sampai n-1
 - Proses linear, satu per satu dikalikan
 - Contoh: $2^3 : 122 \cdot 2 = 8$
 - pangkatDC() : Divide and Conquer
 - Menghitung pangkat dengan rekursi, membagi pangkat menjadi dua
 - Jika pangkat genap : $a^{(n/2)} * a^{(n/2)}$
 - Jika pangkat ganjil : $a^{(n/2)} * a^{(n/2)} * a$
 - Proses logaritmik, lebih cepat untuk pangkat besar
2. Ya, sudah termasuk di pangkatDC()

Contohnya:

```
if(n%2==1){
    return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a); // combine hasil rekursi
}else{
    return (pangkatDC(a, n/2)*pangkatDC(a, n/2)); // combine hasil rekursi
}
```

Di sini hasil dari sub-masalah (pangkatDC(a, n/2)) digabungkan (dikalikan),itu tahap combine pada Divide and Conquer.
3. Saat ini pangkatBF(int a, int n) masih relevan jika ingin menghitung basis dan pangkat berbeda tanpa membuat object baru

Tapi karena class pangkat sudah punya atribut nilai dan pangkat, bisa dibuat tanpa parameter

Contoh versi tanpa parameter:

```
int pangkatBF(){
    int hasil = 1;
    for(int i=0; i<pangkat; i++){
        hasil *= nilai;
    }
    return hasil;
}
```
4. Cara kerja pangkatBF() dan pangkatDC()
 - pangkatBF() → loop / iterasi, mulai dari 1 sampai n, kalikan bertahap

- pangkatDC() → rekursi / divide and conquer, pangkat dibagi 2 setiap langkah, hasil sub-masalah dikalikan saat kembali dari rekursi
- Perbedaan utama: BF linear, DC lebih cepat untuk pangkat besar karena jumlah perkalian lebih sedikit

percobaan 5.4

class sum11.java

```
package BruteForceDivideConquer.minggu5;

public class sum11 {
    double keuntungan[];

    sum11(int el){
        keuntungan = new double[el];
    }

    double totalBF(){
        double total=0;
        for(int i=0;i<keuntungan.length;i++){
            total = total+keuntungan[i];
        }
        return total;
    }

    double totalDC(double arr[], int l, int r){
        if(l==r){
            return arr[l];
        }
        int mid = (l+r)/2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid+1, r);
        return lsum+rsum;
    }
}
```

class mainSum11.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class mainSum11 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("masukkan jumlah elemen: ");
        int elemen = input.nextInt();
    }
}
```

```

sum11 sm = new sum11(elemen);
for(int i=0;i<elemen;i++){
    System.out.print("masukkan keuntungan ke-"+(i+1)+" : ");
    sm.keuntungan[i] = input.nextDouble();
}
System.out.println("total keuntungan menggunakan BruteForce:
"+sm.totalBF());
System.out.println("total keuntungan menggunakan divide and
conquer: "+sm.totalDC(sm.keuntungan, 0, elemen-1));
}
}

```

hasil output:

masukkan jumlah elemen: 5

masukkan keuntungan ke-1: 10

masukkan keuntungan ke-2: 20

masukkan keuntungan ke-3: 30

masukkan keuntungan ke-4: 40

masukkan keuntungan ke-5: 50

total keuntungan menggunakan BruteForce: 150.0

total keuntungan menggunakan divide and conquer: 150.0

pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```

double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);

```

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?
- ```

return lsum+rsum;

```
4. Apakah base case dari totalDC()?
  5. Tarik Kesimpulan tentang cara kerja totalDC()

jawaban

1. mid itu dipakai untuk membagi array jadi dua bagian: kiri dan kanan.  
Misalnya array dari index 0 sampai 5,  $mid = (0+5)/2 = 2$ , berarti kiri [0..2] dan kanan [3..5].  
Tujuannya supaya totalDC bisa menghitung bagian-bagian lebih kecil dulu sebelum digabung.
2. lsum : menghitung jumlah elemen di bagian kiri  
rsum : menghitung jumlah elemen di bagian kanan  
Jadi setiap bagian array dihitung rekursif sampai tinggal satu elemen
3. -Setelah hitung total kiri dan kanan, hasilnya digabung supaya dapat total keseluruhan array.

-Kalau tidak dijumlah, kita cuma dapat salah satu sisi saja, bukan total semua elemen.

```
4. if(l == r){
 return arr[l];
}
```

-Base case terjadi kalau array cuma punya satu elemen

-Di sini rekursi berhenti, dan langsung dikembalikan nilainya

#### 5. Kesimpulan cara kerja totalDC()

- totalDC() bekerja dengan pecah masalah dulu baru gabung hasilnya (Divide and Conquer)
- Langkahnya:
  1. Bagi array menjadi kiri & kanan
  2. Hitung total masing-masing bagian secara rekursif
  3. Gabungkan hasilnya dengan penjumlahan
- Base case : array tinggal satu elemen
- Lebih terstruktur dan rapi dibanding Brute Force, apalagi kalau array besar